

Node.js - SQLite

Zum Persistieren von Daten bietet sich die Verwendung einer Datenbank an. Wir verwenden dazu **SQLite**, eine leichtgewichtige und serverlose Datenbank, die sich gut für kleine bis mittelgroße Anwendungen eignet. SQLite speichert die Daten in einer einzigen Datei, was die Handhabung und Verteilung der Datenbank erleichtert.

Installation

Um SQLite in einem Node.js-Projekt zu verwenden, müssen wir zunächst das entsprechende Paket installieren. Wir verwenden das Paket `sqlite3`, das eine einfache Schnittstelle zur SQLite-Datenbank bietet. Zusätzlich installieren wir `sqlite`, welche Promise-basierte Funktionen bereitstellt. Damit können wir, wie beim `fetch`-API, mit `async` und `await` arbeiten.

```
npm install sqlite3 sqlite
```

Datenbankverbindung herstellen

Wir werden häufig mit einer leeren Datenbank beginnen und beim Programmstart z.B. Daten aus einer CSV- oder JSON-Datei importieren. So können wir beim Programmstart checken, ob die Datenbank-Datei bereits existiert und wenn ja, löschen wir diese:

```
import fs from 'fs';
import sqlite3 from 'sqlite3';
import { open } from 'sqlite';
import path from 'path';

const dbFilePath = path.join(process.cwd(), 'database.sqlite');
if (fs.existsSync(dbFilePath)) {
```

```
    fs.unlinkSync(dbFilePath);  
}
```

Danach können wir die Datenbankverbindung herstellen:

```
const db = await open({  
  filename: dbFilePath,  
  driver: sqlite3.Database  
});
```

Tabellen erstellen

Erstellen wir nun eine Tabelle zum Verwalten von Benutzerdaten mit einer Id und einem Namen:

```
await db.exec(`  
  CREATE TABLE users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL  
  )  
`);
```

Wir werden mit den Datentypen INTEGER für Ganzzahlen und TEXT für Zeichenketten arbeiten. In SQLite gibt es keinen Datentyp für Boolean-Werte oder Datumsangaben, diese werden als INTEGER oder TEXT gespeichert.

Tabellen-Abhängigkeiten definieren

Falls wir Tabellen mit Abhängigkeiten erstellen möchten, können wir Fremdschlüssel verwenden. Zum Beispiel könnten wir eine posts-Tabelle erstellen, die auf die users-Tabelle verweist:

```
await db.exec(`
  CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    title TEXT NOT NULL,
    content TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id)
  )
`);
```

Daten einfügen

Um Daten in die Tabellen einzufügen, verwenden wir INSERT INTO-Anweisungen. Hier ein Beispiel zum Einfügen eines neuen Benutzers:

```
await db.run(`
  INSERT INTO users (name) VALUES (?)
`, ['Max Mustermann']);
```

Man kann hier also mit Platzhaltern (?) arbeiten, um SQL-Injection zu vermeiden. Es besteht aber auch die Möglichkeit, die Werte direkt in die SQL-Anweisung einzufügen, was jedoch nicht empfohlen wird.

Daten abfragen

Um Daten aus der Datenbank abzurufen, verwenden wir SELECT-Anweisungen. Hier ein Beispiel zum Abrufen aller Benutzer:

```
const users = await db.all('SELECT * FROM users');
console.log(users);
```

Ergebnis:

```
[  
  { id: 1, name: 'Max Mustermann' },  
  { id: 2, name: 'Erika Mustermann' }  
]
```

Beachte, hier die Methode `all()` zu verwenden.

Möchten wir nur einen einzelnen Datensatz abrufen, können wir die Methode `get()` verwenden:

```
const user = await db.get('SELECT * FROM users WHERE id = ?', [1]);  
console.log(user);
```

Ergebnis:

```
{ id: 1, name: 'Max Mustermann' }
```

Daten aktualisieren

Um Daten in der Datenbank zu aktualisieren, verwenden wir UPDATE-Anweisungen. Hier ein Beispiel zum Aktualisieren des Namens eines Benutzers:

```
await db.run(`  
  UPDATE users SET name = ? WHERE id = ?  
`, ['Hans Huber', 1]);
```

Betrifft eine Aktualisierung mehrere Datensätze, können wir prüfen, wie viele Datensätze betroffen sind, indem wir den Rückgabewert verwenden. z.B. alle Benutzer, deren Name mit "Mustermann" endet, auf "Muster" ändern:

```
const result = await db.run(`  
  UPDATE users SET name = 'Muster' WHERE name LIKE ?  
`, ['%Mustermann']);
```

```
console.log(`Anzahl der aktualisierten Datensätze:  
↪  ${result.changes}`);
```

Ergebnis:

Anzahl der aktualisierten Datensätze: 2

Daten löschen

Um Daten aus der Datenbank zu löschen, verwenden wir DELETE-Anweisungen. Hier ein Beispiel zum Löschen eines Benutzers:

```
await db.run(`  
  DELETE FROM users WHERE id = ?  
`, [1]);
```

Auch hier können wir die `run()`-Methode verwenden, um die Anzahl der gelöschten Datensätze zu überprüfen:

```
const result = await db.run(`  
  DELETE FROM users WHERE name = ?  
`, ['Muster']);  
console.log(`Anzahl der gelöschten Datensätze: ${result.changes}`);
```

Ergebnis:

Anzahl der gelöschten Datensätze: 1

Datenbankverbindung schließen

Nachdem alle Datenbankoperationen abgeschlossen sind, sollten wir die Verbindung zur Datenbank schließen:

```
await db.close();
```

ACHTUNG: Im Falle einer Express-Anwendung darf die Datenbankverbindung nicht nach jeder Anfrage geschlossen werden, sondern nur beim Beenden der Anwendung.

Übung

Erweitere das **Books-REST-API** so, dass die Daten in einer SQLite-Datenbank gespeichert werden.

Erstelle eine Tabelle `books` mit den Spalten `id`, `title`, `author` und `year` und befülle diese beim Programmstart mit den Daten aus der Datei `books.json`.